

Simulación de Entrevista Técnica — API Gateway & Eureka

Guía de estudio en formato entrevista · Spring Cloud Gateway + Eureka Service Discovery en el sistema POS

Guía de estudio en formato entrevista centrada en el **API Gateway** (Spring Cloud Gateway) y **Eureka** (service discovery) de este proyecto POS. Cada sección incluye la **pregunta del entrevistador**, una **respuesta modelo** y **follow-ups** frecuentes.

Formato sugerido: 45–55 min. Bloques: rol del gateway (10'), Eureka/discovery (15'), routing y filtros (10'), resiliencia (10'), seguridad y operación (10').

0. Warm-up — Gateway y Eureka en el proyecto

P ¿Qué rol cumplen el API Gateway y Eureka aquí?

R El **API Gateway** (:8080 , Spring Cloud Gateway) es el **único punto de entrada**: los tres frontends Angular y clientes externos hablan solo con él, y este enruta a los seis microservicios. Centraliza CORS, rate limiting (con Redis), y agrega la documentación Swagger. **Eureka** (:8761) es el **service discovery**: cada servicio se registra al arrancar y el gateway resuelve destinos por **nombre lógico** (lb://venta-service) en vez de IP/puerto fijos, permitiendo escalar réplicas sin reconfigurar.

1. API Gateway

P1 ¿Qué problemas resuelve un API Gateway en microservicios?

R Evita que el cliente conozca la topología interna (cuántos servicios, dónde están) y centraliza **cross-cutting concerns**: enrutamiento, autenticación/autorización, rate limiting, CORS, TLS termination, agregación de respuestas y observabilidad. Sin gateway, cada frontend tendría que conocer todas las URLs, manejar CORS por servicio y duplicar lógica de seguridad.

FOLLOW-UP ¿Gateway vs BFF (Backend for Frontend)? — El gateway es genérico; un BFF es un gateway especializado por tipo de cliente (web/móvil) que agrega y adapta respuestas a esa UI concreta.

P2 ¿Por qué Spring Cloud Gateway y no Zuul?

R Spring Cloud Gateway está construido sobre **Spring WebFlux/Reactor Netty**: es **no bloqueante y reactivo**, maneja muchas conexiones concurrentes con pocos hilos, ideal para un proxy I/O-bound. Zuul 1 era bloqueante (un hilo por request). Gateway también tiene un modelo limpio de **predicates + filters** y buena integración con Eureka, Resilience4j y el `RedisRateLimiter`.

2. Eureka y service discovery

P3 ¿Cómo funciona el registro y descubrimiento en Eureka?

R Es **client-side discovery**: al arrancar, cada servicio (cliente Eureka) se **registra** en el servidor Eureka con su nombre lógico e instancias (host/puerto). Envía **heartbeats** periódicos (por defecto cada 30s) para renovar el lease. Los clientes (como el gateway) **descargan y cachean** el registro localmente y eligen una instancia con un balanceador cliente (Spring Cloud LoadBalancer). Si un servicio deja de enviar heartbeats, Eureka lo expira del registro.

FOLLOW-UP ¿Client-side vs server-side discovery? — Client-side (Eureka): el cliente conoce las instancias y balancea. Server-side (p. ej. un LB/K8s Service): el cliente llama a una VIP y el LB balancea. En K8s el DNS del cluster + Service cumple este rol, por eso Eureka podría omitirse allí.

P4 ¿Qué es el "self-preservation mode" de Eureka?

R Cuando Eureka detecta que **demasiados** servicios dejaron de renovar el lease en poco tiempo, asume que el problema es de **red** (partición) y no que todos cayeron, así que **deja de expulsar** instancias para no vaciar el registro por error. Prioriza disponibilidad sobre consistencia (Eureka es un sistema **AP** en términos de CAP).

P5 ¿Qué pasa si Eureka se cae?

R Hay **degradación elegante**: los clientes cachean el último registro conocido, así que siguen resolviendo servicios por un tiempo. No se descubren instancias *nuevas* mientras Eureka esté caído. Por eso en producción se corren **varias réplicas de Eureka** (peer-to-peer, replicándose entre sí) en distintas zonas.

3. Routing, filtros y load balancing

P6 Explica predicates y filters en Spring Cloud Gateway.

R Una **route** = predicate(s) + uri + filter(s). Los **predicates** deciden si la ruta aplica (por path, método, host, header, etc.). Los **filters** modifican request/response (reescribir path, añadir cabeceras, rate limit, circuit breaker).

```
spring:
  cloud:
    gateway:
      routes:
        - id: venta-service
          uri: lb://venta-service
          predicates:
            - Path=/api/ventas/**
          filters:
            - StripPrefix=1
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 10
                redis-rate-limiter.burstCapacity: 20
```

`lb://` indica al gateway que resuelva `venta-service` vía discovery y balancee.

P7 ¿Cómo balancea carga entre réplicas?

R Con **Spring Cloud LoadBalancer** (client-side). El gateway obtiene la lista de instancias desde Eureka y elige una (round-robin por defecto). Al añadir réplicas, el gateway las ve automáticamente sin reconfiguración; al morir una, sale del registro y deja de recibir tráfico.

4. Resiliencia

P8 ¿Cómo evitas que un servicio lento tumbe al gateway?

R Con **timeouts** por ruta y un **circuit breaker** (Resilience4j): el filtro `CircuitBreaker` abre el circuito tras un umbral de fallos/lentitud y devuelve un **fallback** rápido en vez de encolar peticiones. Combinado con **bulkheads** (aislar pools) y **retries** acotados con backoff, evita fallos en cascada.

FOLLOW-UP ¿Estados de un circuit breaker? — **Closed** (pasa tráfico), **Open** (rechaza y devuelve fallback), **Half-Open** (deja pasar unas pocas pruebas para ver si el servicio se recuperó).

5. Seguridad y operación

P9 ¿Dónde validas el JWT: en el gateway o en cada servicio?

R Se puede validar la firma en el **gateway** (rechazo temprano de tokens inválidos, menos carga en servicios) y **además** en los servicios como defensa en profundidad (no confiar solo en el borde). El gateway propaga el token o claims (p. ej. cabeceras) hacia los servicios. También centraliza **CORS** y el **rate limiting** del login.

P10 ¿Cómo agrega el gateway la documentación Swagger?

R Cada servicio expone su OpenAPI (`/api-docs`) y el gateway ofrece un Swagger UI **agregado** en `http://localhost:8080/swagger-ui.html` que lista los servicios. Así hay un único portal de documentación para todo el sistema.

6. Diseño abierto / escenarios

P11 Migras a Kubernetes. ¿Mantienes Eureka?

R Es opcional. En K8s el **Service + DNS del cluster** ya hacen discovery y balanceo server-side (`venta-service.namespace.svc`), y un **Ingress** puede sustituir parte del gateway. Se mantiene Eureka por **portabilidad** (que el mismo stack corra igual en Docker Compose y K8s) y para no reescribir la resolución `lb://` . La alternativa es quitar Eureka y usar DNS nativo, simplificando la operación en K8s.

P12 El gateway es un único punto de entrada: ¿no es un SPOF?

R Lo sería con una sola instancia. Se mitiga corriendo **varias réplicas** del gateway detrás de un balanceador/Ingress, sin estado local (el estado de rate limit vive en Redis, compartido). Así cualquier réplica atiende cualquier request y la caída de una no afecta al servicio.

Apéndice — Chuleta rápida

Tema	Punto clave
Gateway	Único punto de entrada; Spring Cloud Gateway sobre WebFlux (reactivo)
Cross-cutting	Routing, CORS, rate limiting (Redis), JWT, Swagger agregado
Eureka	Client-side discovery; registro + heartbeats (30s); resuelve <code>lb://</code>
CAP	Eureka es AP; self-preservation evita vaciar el registro en particiones
Eureka caído	Degradación elegante (clientes cachean); replicar en HA peer-to-peer
Routing	Route = predicates + uri (<code>lb://</code>) + filters
Balanceo	Spring Cloud LoadBalancer (client-side), round-robin sobre instancias
Resiliencia	Timeouts + Resilience4j (circuit breaker closed/open/half-open) + fallback
K8s	Service+DNS+Ingress pueden reemplazar Eureka/gateway; se mantiene por portabilidad